INTRODUCEREA CETEI IN IMAGINI

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

MODELAREA CEȚEI(1)

Ceaţa adaugă realism imaginii şi ajută în eliminarea defectelor produse de decuparea la nivelul planului din spate al volumului vizual:

- Fără ceaţă, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se aproprie de planul din spate şi este decupat atunci când planul din spate îl intersectează.
- Cu ceaţă, dacă intensitatea ceţii creşte proporţional cu distanţa de la poziţia observatorului, se crează efectul de distanţă pentru obiectele îndepărtate, iar obiectele decupate la nivelul planului din spate nu mai sunt vizibile.

Cfog – culoarea de ceaţă

Clocal – culoarea calculată într-un punct folosind modelul de iluminare locala

$0 <= \Phi <= 1$ – factorul de ceaţă, proporţional cu distanţa de la observator la punct

($\Phi=1$: opacitate)

Culoarea punctului, afectata de ceata: **$C_{loc_fog} = (1-\Phi)*C_{local} + \Phi*C_{fog}$**

MODELAREA CETEI(2)

Fie dist distanta de la observator la un punct din scena vizualizata.

Factorul de ceaţă liniar

- Actioneaza intre 2 distante definite in sistemul coordonatelor observator, z0 si z1, masurate pe directia in care priveste observatorul.

  $\Phi = 0$ pentru dist < z0

  $\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

  $\Phi = 1$ pentru dist > z1

Factorii de ceaţă exponentiali

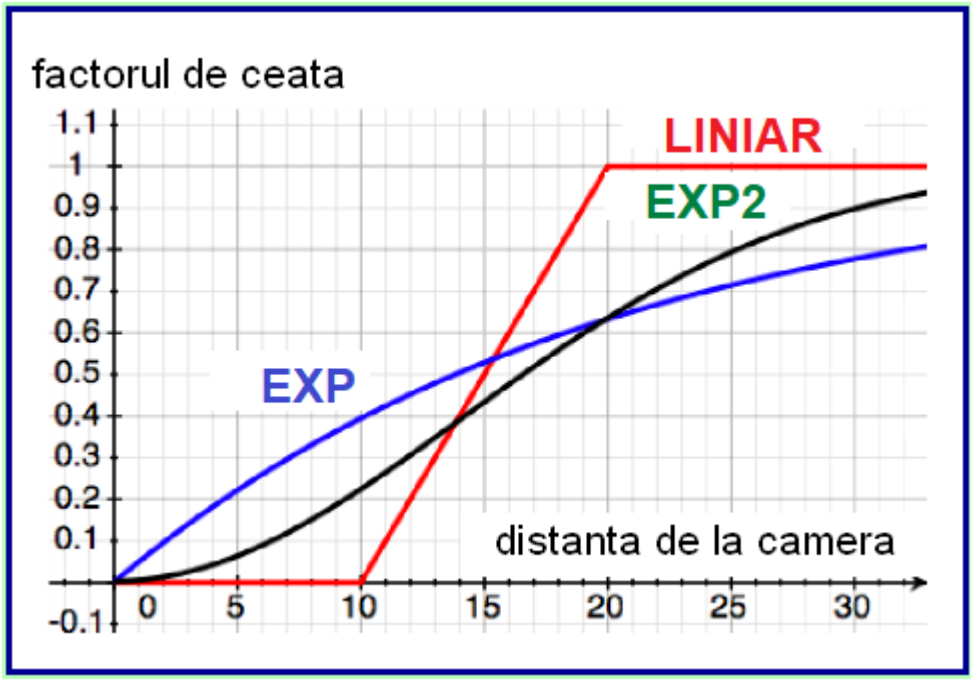
$\Phi = e^f$

EXP: $f = (- fog_density * dist)$

EXP2: $f = (- fog_density * dist)^2$

- fog_density: densitatea cetii (un numar real, de ex. 0.05) - modeleaza atenuarea luminii datorata cetii

MODELAREA CETEI(3)



z0 = 10; z1 = 20; fog_density = 0.3

INTRODUCEREA CETEI IN IMAGINI (1)

Includerea cetei in calculul culorii se poate efectua:

- In vertex shader (modelul Gouraud):

- se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia varfului transformata in coordonate observator

- se calculeaza culoarea varfului afectata de ceata: **Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog**

- **Cloc_fog** este iesire din vertex shader

- culorile afectate de ceata ale varfurilor sunt interpolate de GPU la nivel de fragmente

- culoarea de ceață interpolata, **Cloc_fog,** este intrare pentru fragment shader

- In fragment shader(modelul Phong):

Varianta (1)

- se transmite de la vertex shader pozitia varfului in coordonate observator

- pozitiile varfurilor in coordonate observator sunt interpolate de GPU la nivel de fragmente

- in fragment shader se calculeaza **Φ** folosind distanta (dist) de la observator la pozitia fragmentului in coordonate observator

INTRODUCEREA CETEI IN IMAGINI (2)

```
// Calculul culorii de ceata in fragment shader- Varianta (1)

//Vertex shader
layout(location = 0) in vec3 in_position;
layout(location = 1) in vec3 in_normal;
uniform mat4 model_matrix, view_matrix, projection_matrix;

out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala transformata in coord globale
out vec4 view_pos; //pozitia varfului in coord observator

void main(){
  world_pos = (model_matrix * vec4(in_position,1)).xyz;
  world_normal = normalize(mat3(model_matrix) * in_normal);
  view_pos = view_matrix * model_matrix * vec4(in_position,1);
  gl_Position = projection_matrix * view_pos;
 }
```

```
//fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform int fogSelector; //0 linear; 1 exponential; 2 exponential square
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
……..
in vec3 world_pos;// pozitia interpolata a fragmentului in coord globale
in vec3 world_normal;/ normala in coord globale interpolata
in vec4 view_pos;// pozitia interpolata a fragmentului in coord observator
void main(){
//Calculeaza Cfragment aplicand modelul de iluminare locala in pozitia fragmentului(world_pos)
   Cfragment = ….
//Calculeaza distanta de la pozitia interpolata a fragmentului la observator
dist = length(view_pos); //vectorul (0,0,0) → view_pos
if(fogSelector == 0)//linear fog
{  fogFactor = (dist - 20)/(80 - 20); // z0 =20; z1=80
  fogFactor = clamp( fogFactor, 0.0, 1.0 );//restrange fogFactor la intervalul 0-1
  out_color = (1-fogFactor)*Cfragment + fogFactor*Cfog
} ……..}
```

INTRODUCEREA CETEI IN IMAGINI (3)

```
// Calculul culorii de ceata in fragment shader- Varianta (2)
//Vertex shader
out vec3 world_pos;// pozitia varfului in coord globale
out vec3 world_normal;//normala transformata in coord globale
//fragment shader
layout(location = 0) out vec3 out_color;
uniform vec3 light_position;
uniform vec3 eye_position;
uniform vec3 Cfog = vec3(0.5, 0.5,0.5);//culoarea cetii
in vec3 world_pos;// pozitia interpolata a fragmentului in coord globale
in vec3 world_normal;// normala in coord globale interpolata
void main(){
//Calculeaza Cfragment  aplicand modelul de iluminare locala in pozitia fragmentului (world_pos)
   Cfragment = ….
// Calculeaza pozitia fragmentului in coordonate observator: frg_pos → frg_eye_pos
//Calculeaza distanta de la pozitia fragmentului, frg_eye_pos, la observator
   dist = length(frg_eye_pos);
//Calculeaza fogFactor folosind dist
…….
```

INTRODUCEREA CETEI IN IMAGINI (4)

Calculeaza pozitia fragmentului in coordonate observator: frg_pos → frg_eye_pos

//transformarea in viewport

gl.FragCoord.x = (width/2)frg_xd + ox , ox = px+ width/2

gl.FragCoord.y = (height/2)frg_yd + oy, oy = py + height/2

gl.FragCoord.z = ((f-n)/2)frg_zd + (n+f)/2 = frg_zd/2 + 0.5 // implicit: n=0, f=1

Rezulta →

frg_xd = (gl.FragCoord.x – ox)/(width/2)

frg_yd = (gl.FragCoord.y – oy)/(height/2)

frg_zd = (gl.FragCoord.z – 0.5)*2

//transformarea de normalizare

frg_ xd = frg_ xc/frg_wc; frg_ yd = frg_ yc/frg_wc; frg_ zd = frg_ zc/frg_wc; frg_ wc = -eye

//pozitia fragmentului in coordonate de decupare

frg_clip_pos = [frg_xc, frg_yc, frg_zc, fgr_wc]^T

//transformarea de proiectie

frg_clip_pos = projection_matrix * frg_eye_pos



//pozitia fragmentului in coordonate observator

frg_eye_pos = (projection_matrix)$^{-1}$ *frg_clip_pos